KRR Lab 1 - Structural Subsumption in Description Logics (FL_0)

Summary of the theory and algorithm

What are Description Logics?

Description Logics (DLs) are a family of formal knowledge representation languages used to represent and reason about structured knowledge in a precise, unambiguous way.

What are they used for?

- **Ontology modeling** (e.g., medical terminologies, semantic web)
- **Database schema design** (conceptual modeling)
- **Intelligent information systems** (classification, query answering)
- Foundation of **OWL** (Web Ontology Language)

Example Use Case

Modeling a university domain:

- **Concepts**: Student, Professor, Course, PhDStudent
- **Relationships**: teaches, enrolledIn, advises
- **Reasoning**: "Is a PhDStudent a type of Student?" (subsumption)

Key Idea

DLs provide a formal way to define concepts and check logical relationships between them automatically.

DL Operators and Symbols

Description Logics use a set of **constructors** to build complex concept descriptions from simpler ones.

Basic Elements

- **Concept names**: A, B, C, … (e.g., Student, Professor)
- **Role names**: R, S, T, … (e.g., teaches, enrolledIn)
- **Top concept**: $\top$ (represents everything)
- **Bottom concept**: $\bot$ (represents nothing/empty set)

Main Constructors

Constructor	Syntax	Meaning
Conjunction	$C \sqcap D$	Intersection of concepts C and D
Disjunction	$C \sqcup D$	Union of concepts C and D
Negation	$\neg C$	Complement of concept C
Universal restriction	$\forall R.C$	All R-successors satisfy C
Existential restriction	$\exists R.C$	At least one R-successor satisfies C
Number restrictions	$\geq nR$, $\leq nR$	At least/most n R-successors

Reading and Interpreting DL Expressions

How to Read DL Expressions

Example 1: `Professor ⊓ ∀teaches.GraduateCourse`

- **Reading**: "A Professor who teaches only Graduate Courses"
- **Interpretation**: Any individual that is both a Professor AND all courses they teach are Graduate Courses

Example 2: `Student ⊓ ∃enrolledIn.AIcourse`

- **Reading**: "A Student enrolled in at least one AI Course"
- **Interpretation**: Any individual that is a Student AND is enrolled in at least one AI course

Example 3: `∀hasChild.(Doctor ⊓ ∀specializes.Cardiology)`

- **Reading**: "Someone whose children are all Cardiologists"
- **Interpretation**: All children of this individual are Doctors specialized in Cardiology

Key Insight

- ∀R.C means "if there are R-successors, they ALL must be C"
- ∃R.C means "there EXISTS at least one R-successor that is C"

Classification of DL Languages

DL languages are classified by which constructors they allow. Each language has a **name** formed by letters indicating its features.

Common DL Languages

Language	Allowed Constructors	Expressivity
FL_0	⊓, ∀R.C	Very restricted (polynomial reasoning)
FL⊥	FL_0 + ⊥	Adds unsatisfiability
AL	⊓, ∀R.C, ¬A, ⊤, ⊥	Adds atomic negation
ALC	AL + ¬C	Adds full negation (propositional modal logic K)
ALCN	ALC + ≥nR, ≤nR	Adds number restrictions
SHOIN	Very expressive	Basis for OWL DL

Naming Convention

- **AL** = Attributive Language (base)
- **C** = full negation (Complement)
- **N** = Number restrictions
- **S** = role transitivity (S_4 modal logic)
- **H** = role Hierarchy
- **I** = Inverse roles
- **O** = nOminals (individuals)

Trade-off: More expressive → higher computational complexity

The FL_0 Language

FL_0 is one of the simplest DLs, allowing only:

- **Conjunction** ($\sqcap$)
- **Universal restriction** ($\forall R.C$)
- **Concept names** (A, B, C, ...)
- **Top concept** ($\top$, implicitly)

Normal Form for FL_0

Any FL_0 concept can be written as:

$$A_1 \sqcap A_2 \sqcap \ldots \sqcap A_m \sqcap \forall R_1.C_1 \sqcap \forall R_2.C_2 \sqcap \ldots \sqcap \forall R_n.C_n$$

Where:

- $A_1, \ldots, A_m$ are **distinct concept names**
- $R_1, \ldots, R_n$ are **distinct role names**
- $C_1, \ldots, C_n$ are FL_0 concepts **in normal form** (recursively)

Example Normalization

$\forall R.(A \sqcap B) \sqcap A$ is already in normal form

$\forall R.A \sqcap \forall R.B \sqcap A$ → normalizes to → $A \sqcap \forall R.(A \sqcap B)$

Structural Subsumption Algorithm for FL_0

Goal: Determine if concept C is subsumed by concept D (written $C \sqsubseteq D$)

Meaning: $C \sqsubseteq D$ means "every instance of C is also an instance of D"

Algorithm Steps

Step 1: Normalize both concepts C and D into normal form

```
C = A₁ ⊓ ... ⊓ Aₘ ⊓ ∀R₁.C₁ ⊓ ... ⊓ ∀Rₙ.Cₙ
D = B₁ ⊓ ... ⊓ Bₖ ⊓ ∀S₁.D₁ ⊓ ... ⊓ ∀Sₗ.Dₗ
```

Step 2: Check two conditions:

(i) Concept name check: For every concept name B_i in D, there must exist a matching concept name A_j in C (i.e., $B_i = A_j$)

(ii) Role restriction check: For every $\forall S_i.D_i$ in D, there must exist $\forall R_j.C_j$ in C such that:

- $S_i = R_j$ (same role), AND
- $C_j \sqsubseteq D_i$ (recursive subsumption check)

Result: $C \sqsubseteq D$ **if and only if** both conditions hold

Complexity

Polynomial time in the size of the concept descriptions!

Example - Structural Subsumption in Action

Question

Is `A ⊓ B ⊓ ∀R.(C ⊓ D) ⊓ ∀S.E` subsumed by `A ⊓ ∀R.C`?

Solution

Step 1: Both already in normal form

- Expr1 = `A ⊓ B ⊓ ∀R.(C ⊓ D) ⊓ ∀S.E`
- Expr2 = `A ⊓ ∀R.C`

Step 2: Check conditions

(i) Concept names in Expr2: {A}

- Is A in Expr1? ✓ Yes

(ii) Role restrictions in Expr2: {∀R.C}

- Does Expr1 have ∀R.something? ✓ Yes: ∀R.(C ⊓ D)
- Is `(C ⊓ D) ⊑ C`? ✓ Yes (anything conjoined with C is more specific than C)

Conclusion: ✓ `A ⊓ B ⊓ ∀R.(C ⊓ D) ⊓ ∀S.E ⊑ A ⊓ ∀R.C`

Intuition

Expr1 is more specific (has more constraints: B, ∀S.E, and D in the role restriction), so it's subsumed by the more general Expr2.

Summary & Next Steps

What You Learned

- Description Logics provide formal knowledge representation
- FL_0 is a simple but useful DL with conjunction and universal restrictions
- Structural subsumption uses syntactic comparison of normal forms
- The algorithm is recursive and efficient (polynomial time)

Your Task

Implement the structural subsumption algorithm in Python:

1. Represent FL_0 concepts as data structures
2. Implement normalization (optional, if time permits)
3. Implement the subsumption checking algorithm
4. Test on provided examples of increasing complexity

Resources

- Provided Python skeleton code with test cases